Yazılım Tasarımı: Özellik Etkileşimi Problemi

Ömer Faruk Yavuz

November 2026

Özet

Özellik etkileşimi problemi (*feature interaction problem*), tek başına doğru çalışan iki ya da daha fazla kuralın birlikte devreye girdiğinde sistemi yanlış davranışa götürmesidir. Kusur kurallardan birinde değil, kesişimlerinde bulunur; bu nedenle kuralları tek tek sınamak onu göstermez. Terim 1980'lerin sonunda telefon santrali yazılımlarından çıkmıştır (Bowen vd., 1989). Etkileşimin iki ana türü vardır: iki kuralın aynı olaya sahip çıkması ve bir kuralın ötekinin varsayımını bozması. İkinci tür hiçbir açık çakışma üretmediği için daha zor fark edilir. Kesişim sayısı kural sayısıyla hızla büyür: on kural 45 ikili ve 120 üçlü kesişim üretir. Test kesişimi bulur, fakat hangi kuralın kazanacağına karar vermez; bu karar tasarımda, kurallar arasına yazılan açık bir öncelik sırasıyla verilir. Sıra yazılmadığında sonucu kodun çalışma sırası belirler.

Giriş

Bir yazılım ürünü büyüdükçe davranışı tek bir algoritmayla değil, üst üste eklenen kurallarla belirlenir. Burada **kural** ya da **özellik** (*feature*), belirli bir koşulda sistemin çıktısını değiştiren bağımsız bir davranış birimidir. "Sessiz kısımları at" bir kuraldır; "konuşmacı varsa en az bir *soundbite* bulunsun" başka bir kuraldır.

Her kural genellikle ayrı bir zamanda, çoğu kez ayrı bir kişi tarafından yazılır ve kendi testleriyle doğrulanır. Yaygın kabul, her biri testten geçen kuralların toplamının da doğru çalışacağıdır. Bu kabul yanlıştır. İki kural aynı girdide birlikte devreye girdiğinde, ikisinin de yazarının öngörmediği bir durum oluşabilir. **Özellik etkileşimi**, bir kuralın varlığının başka bir kuralın davranışını değiştirmesidir. Etkileşim istenmeyen bir sonuç ürettiğinde problemden söz edilir.

Her etkileşim hata değildir. Bazı kuralların birbirini etkilemesi tasarımın parçasıdır (Zave, 1993). Problem, hakkında kimsenin karar vermediği etkileşimlerdir.

Bu not kural tabanlı yazılımlardaki istenmeyen etkileşimleri kapsar: nasıl ortaya çıktıklarını, neden zor bulunduklarını, hangi testlerle yakalandıklarını ve tasarımda nasıl çözüldüklerini. Donanım düzeyindeki etkileşimler ve biçimsel doğrulama yöntemleri kapsam dışındadır.

Kökeni ve Klasik Örnek

Telefon Santrallerinden Gelen Ad

Terim telekomünikasyon yazılımlarından gelir. 1980'lerde telefon şirketleri santrallere arama yönlendirme, bekleyen arama, arama engelleme gibi yüzlerce abone özelliği ekledi. Her özellik ayrı bir ekip tarafından geliştiriliyordu. Özellik sayısı arttıkça, ayrı ayrı doğru çalışan özelliklerin birlikte açıldığında hatalı davrandığı görüldü. Bowen ve arkadaşları (1989) bu durumu ayrı bir problem olarak adlandırdı. Konuya ayrılmış uluslararası çalıştay dizisi (*Feature Interaction Workshop*, FIW) 1992'de başladı. Calder ve arkadaşları (2003) bu alandaki çalışmaların kapsamlı bir değerlendirmesini sunar.

Yönlendirme ile Bekleyen Arama

Alanın standart örneği iki özellikten oluşur (Cameron vd., 1993):

- **Meşgulde arama yönlendirme**: Abone meşgulken gelen arama başka bir numaraya aktarılır.
- **Bekleyen arama**: Abone meşgulken bir uyarı sesi duyar ve ikinci aramayı alabilir.

İki özellik de açıkken abone meşgul durumdayken biri aradığında santralin ne yapacağı belirsizdir. İki özellik de aynı olaya, yani meşgul hatta gelen aramaya sahip çıkar. İkisi de kendi tanımına göre doğrudur. Hata, iki ekibin birbirinin kuralını hesaba katmamış olmasındadır.

Etkileşim Türleri

Aynı Olaya Sahip Çıkma

İlk türde iki kural aynı tetikleyiciyle devreye girer ve aynı çıktıyı farklı yönde değiştirmek ister. Yönlendirme ile bekleyen arama bu türdendir. Bu tür görece kolay bulunur, çünkü iki kuralın tetikleyicisi ortaktır ve çakışma açıkça görülür.

Varsayımın Bozulması

İkinci türde kurallar aynı olaya sahip çıkmaz. Bir kural, ötekinin doğru çalışmak için dayandığı bir varsayımı geçersiz kılar. Telekomünikasyondaki örnek şudur (Cameron vd., 1993): A abonesi C numarasına yapılan aramaları engellemiştir. A, B'yi arar. B'nin yönlendirmesi aramayı C'ye aktarır. Engel delinmiştir. Engelleme kuralı "aranan numara, bağlanılan numaradır" varsayımına dayanıyordu; yönlendirme bu varsayımı bozmuştur. Hiçbir aşamada iki kural aynı kararı vermeye çalışmamıştır, bu yüzden ortada görünür bir çakışma yoktur.

Yayın Paketi Üretiminden Örnekler

Aynı yapı, bir video ya da ses kaydından otomatik haber paketi üreten bir üründe de görülür:

- "Sessiz kısımları at" kuralı ile "konuşmacı varsa en az bir *soundbite* bulunsun" kuralı. Birinci kural kırpar, ikinci kural korumayı sağlayamaz ve paket sıfır *soundbite* ile çıkar.
- Tekrar gösterim (*replay*) kuralı ile atlama kuralının aynı bölüm üzerinde farklı karar vermesi.
- İsim bandı ile altyazının aynı anda ekranın alt bölümüne yerleşmeye çalışması. Burada çakışan şey bir karar değil, ortak bir kaynaktır: ekran alanı.

Problemin Zorluğu

Kesişim Sayısı

Kesişimlerin sayısı kural sayısından çok daha hızlı büyür. On kural için ikili kesişim sayısı $10 \cdot 9/2 = 45$, üçlü kesişim sayısı $10 \cdot 9 \cdot 8/6 = 120$ olur. Yirmi kuralda ikili sayısı 190'a çıkar. Bu kesişimlerin tamamını düşünerek bulmak mümkün değildir.

Tekil Testlerin Körlüğü

Bir kuralın kendi testi, yalnızca o kuralın tek başına devrede olduğu durumu sınar. Kusur iki kuralın birlikte devrede olduğu durumda bulunduğu için, tekil testlerin tamamı geçse bile kusur görünmez kalır. Testlerin sayısını artırmak bunu değiştirmez; değişmesi gereken, testlerin sınadığı durumdur.

Her Yeni Kuralın Maliyeti

Yeni eklenen her kural, mevcut bütün kurallarla yeni kesişimler oluşturur. On kurallı bir sisteme on birinci kural eklendiğinde on yeni ikili kesişim doğar. Bu nedenle etkileşim denetimi bir kerelik bir iş değildir; kural eklemenin sabit bir adımıdır.

Test ile Yakalama

Dört test türü bu problemin farklı yönlerini karşılar.

Test türü	Etkileşim problemindeki işlevi
Kombinasyon testi	Kesişimleri sistematik olarak açar ve dener
Değişmez denetimi	Hangi kural kazanırsa kazansın sonucun bozulduğu durumu yakalar
Metamorfik test	Önemsiz bir girdi değişikliğinin başka bir kuralı devreye sokmasını yakalar
Onaylı referans	Yeni bir çakışmanın eski doğru sonucu bozduğunu gösterir

Kombinasyon Testi

Kombinasyon testi (*combinatorial testing*), kuralların açık ve kapalı durumlarını belirli bir düzende birleştirerek sınar. En yaygın biçimi ikili testtir (*pairwise testing*): her kural çiftinin her durum birleşimi en az bir testte yer alır. Kuhn ve arkadaşları (2004), incelenen yazılım hatalarının büyük bölümünün tek bir etkenle ya da iki etkenin birleşimiyle tetiklendiğini, daha fazla etken gerektiren hataların giderek azaldığını göstermiştir. İkili test bu nedenle verimlidir. Sınırı da buradan gelir: üç ya da daha fazla kuralın birlikte gerektirdiği çakışmalar ikili testte gözden kaçabilir.

Değişmez Denetimi

Değişmez (*invariant*), hangi kurallar devreye girerse girsin çıktıda her zaman doğru olması gereken koşuldur. "Konuşmacı içeren bir kayıttan üretilen pakette en az bir *soundbite* bulunur" bir değişmezdir. Değişmez denetimi doğru sonucu garanti etmez; ihlali yakalar. Garanti, kuralın kodda zorlanmasıyla sağlanır. Değişmezin değeri, hangi kural çiftinin soruna yol açtığını önceden bilmeyi gerektirmemesidir.

Metamorfik Test

Metamorfik test (*metamorphic testing*), tek bir çıktının doğruluğunu değil, iki çalıştırma arasındaki ilişkiyi sınar (Chen vd., 1998). Girdi, sonucu değiştirmemesi gereken bir biçimde değiştirilir ve çıktının aynı kaldığı denetlenir. Örneğin kaydın başına bir saniyelik sessizlik eklemek seçilen *soundbite* kümesini değiştirmemelidir. Değiştiriyorsa, önemsiz bir girdi farkı başka bir kuralı devreye sokmuştur.

Onaylı Referans

Onaylı referans testi (*golden master* ya da *approval testing*), bir kez elle incelenip doğru kabul edilen çıktıyı saklar ve sonraki her çalıştırmayı onunla karşılaştırır. Yeni eklenen bir kural mevcut bir sonucu bozduğunda fark hemen görünür. Bu test neyin doğru olduğunu söylemez; neyin değiştiğini söyler.

Tasarımda Çözüm

Öncelik Sırası

Test kesişimi bulur, fakat "hangi kural kazanır" sorusunu cevaplamaz. Bu soru tasarımda cevaplanır: kurallar arasına açık bir öncelik sırası yazılır. Örneğin "*soundbite* koruma, sessizlik kırpmadan önce gelir." Sıra yazılmadığında iki kural çakıştığında hangisinin uygulanacağı kodun çalışma sırasına kalır. Bu sıra çoğu kez bilinçli bir karar değil, kuralların eklenme tarihinin sonucudur.

Öncelik Dışındaki Çözümler

Öncelik sırası tek çözüm değildir. Bazı çakışmalarda bir kuralın kazanması iki tarafı da tatmin etmez. Bu durumda iki yol kalır:

- **Karşılıklı dışlama**: İki kuralın birlikte açılması yasaklanır.
- **Birleştirme**: İki kural, ortak durumu açıkça ele alan tek bir kurala dönüştürülür. İsim bandı ile altyazı için tek bir yerleşim kuralı bunun örneğidir.

Aday Çiftlerin Daraltılması

Bütün kesişimleri incelemek gerekmez. Her kural için iki bilgi yazılır: kuralı neyin tetiklediği ve kuralın neyi değiştirdiği. Tetikleyicisi ya da değiştirdiği çıktı ortak olan kurallar aday çifttir. Ortak noktası olmayan kurallar elenir. Bu denetim varsayım bozulması türünü tam yakalamaz, çünkü o türde ortaklık dolaylıdır; yine de aday sayısını önemli ölçüde azaltır.

Tek Karar Noktası

Her kural çıktıyı kendi başına değiştirdiğinde öncelik sırası kâğıtta kalır; sonucu yine çalışma sırası belirler. Sıranın gerçekten uygulanması için kurallar çıktıyı değiştirmez, yalnızca öneri üretir. Son kararı öncelik sırasını bilen tek bir nokta verir.

Kazanan Kuralın Kaydı

Her çıktı için hangi kuralların devreye girdiği ve hangisinin kazandığı kaydedilir. Bu kayıt olmadan hatalı bir çıktının hangi çakışmadan doğduğunu sonradan bulmak çok zordur.

Sonuç

Kural tabanlı bir sistemde doğruluk, kuralların tek tek doğruluğundan türetilemez. Bunun pratik karşılığı, test ve tasarım emeğinin bir bölümünün kuralların kendisinden kesişimlerine kaydırılmasıdır. Yeni bir kural eklerken sorulacak soru yalnızca "bu kural doğru mu" değil, "bu kural hangi mevcut kuralla aynı tetikleyiciyi ya da aynı çıktıyı paylaşıyor" sorusudur.

Test ile tasarım burada farklı işler görür ve biri ötekinin yerini tutmaz. Test çakışmanın varlığını gösterir; çakışmada neyin doğru olduğuna ürün kararı verir. Öncelik sırası yazılmamış bir sistemde testin bulduğu her çakışma, aslında verilmemiş bir ürün kararını ortaya çıkarır.

Çerçevenin sınırları vardır. Tetikleyici ve çıktı karşılaştırması varsayım bozulması türünü tam yakalamaz. İkili kombinasyon testi üç ve daha fazla kuralın birlikte gerektirdiği çakışmaları kaçırabilir. Kural sayısı çok büyüdüğünde öncelik sırasının kendisi de bakımı zor bir yapıya dönüşür.

Açık kalan soru, bir kuralın dayandığı örtük varsayımların nasıl görünür kılınacağıdır. Tetikleyici ve çıktı yazılabilir; "aranan numara bağlanılan numaradır" türü varsayımlar ise çoğu kez ancak bozulduklarında fark edilir.

Kaynaklar

- Bowen, T. F., Dworack, F. S., Chow, C. H., Griffeth, N., Herman, G. E. ve Lin, Y.-J. (1989). The feature interaction problem in telecommunications systems. *Seventh International Conference on Software Engineering for Telecommunication Switching Systems*.

- Calder, M., Kolberg, M., Magill, E. H. ve Reiff-Marganiec, S. (2003). Feature interaction: a critical review and considered forecast. *Computer Networks*, 41(1).

- Cameron, E. J., Griffeth, N. D., Lin, Y.-J., Nilson, M. E., Schnure, W. K. ve Velthuijsen, H. (1993). A feature-interaction benchmark for IN and beyond. *IEEE Communications Magazine*, 31(3).

- Chen, T. Y., Cheung, S. C. ve Yiu, S. M. (1998). Metamorphic testing: a new approach for generating next test cases. Teknik rapor HKUST-CS98-01, Hong Kong University of Science and Technology.

- Kuhn, D. R., Wallace, D. R. ve Gallo, A. M. (2004). Software fault interactions and implications for software testing. *IEEE Transactions on Software Engineering*, 30(6).

- Zave, P. (1993). Feature interactions and formal specifications in telecommunications. *IEEE Computer*, 26(8).